



2D physics

Overview

Chapter 8 used Arcade Physics to find out when things touch. This chapter is about the physics itself: velocity and acceleration, gravity, bounce, drag and damping, mass, friction, the kinds of body, rotation, speed limits and world wrapping - and, briefly, Matter.js. The big idea to land is that **a physics body is a set of numbers the engine integrates for you**: students set a velocity or an acceleration and stop writing `x += speed * delta`. The playground makes every setting something to press and watch; the space ship shows the same settings adding up to a game; the Matter project shows what Arcade cannot do.

The chapter's one real trap is new, and silent: **adding a body to a physics group resets it** to the group's defaults. Expect "my ball doesn't move" in every lab until students have met it.

Prerequisites

- Chapter 8: bodies, `physics.add.existing`, `setCircle`, `collider` vs `overlap`, groups, callbacks cast with `as`, `disableBody` / `enableBody`, process callbacks
- Chapter 7: particles and the explosion animation (used, not taught, in the space ship)
- Chapter 2: scenes are reused, so fields reset in `init()`
- school physics: speed, acceleration, momentum. Worth five minutes of revision for some groups

Learning outcomes

Students can:

1. explain the difference between velocity and acceleration, and set both on an Arcade body
2. use world and body gravity, bounce, world bounds, linear drag and damping, and explain why drag has no effect while a body accelerates

3. choose between dynamic, immovable, non-pushable and static bodies, and say what mass and Arcade friction do
4. read `body.blocked` / `body.touching` , and listen for `"worldbounds"`
5. make a ship turn with angular acceleration and fly the way it points with `velocityFromRotation` , with a true top speed (`setMaxSpeed`) and wrapping (`world.wrap`)
6. explain the fixed time step, and pause or slow the physics world
7. make a simple Matter.js scene with shaped bodies, and say when Matter is the better choice

Suggested session plan (2 x 1 hour lab, or 1 hour lecture + 2 hour lab)

Time	Activity
0:00 - 0:10	Run <code>ch09_physics_playground</code> . Let students press every key for five minutes before any explanation. Ask: "what did D do? why do crates not bounce?"
0:10 - 0:25	Slides 3-6: the physics step, velocity vs acceleration, gravity and bounce, drag vs damping (draw the two curves on the board before showing the diagram)
0:25 - 0:35	Live demo of the group trap (below). Slide 7
0:35 - 0:50	Slides 8-11: kinds of body, friction on the moving platform, <code>blocked</code> / <code>touching</code> , pause and <code>timeScale</code> , the fixed step
0:50 - 1:00	Challenges 1 and 2
1:00 - 1:10	Fly <code>ch09_space_ship</code> . Ask: "why does the ship keep going? why does it turn with a little lag?"
1:10 - 1:30	Slides 12-15: angles and facing, angular acceleration, thrust with <code>velocityFromRotation</code> , damping and <code>setMaxSpeed</code> , wrap
1:30 - 1:40	Slides 16-18: Matter.js - drag the tower over, fire cannonballs; Arcade vs Matter
1:40 - 2:00	Challenges 3 and 4 (5 and 6 for the fast ones, or as homework)

Key points to stress

- **physics does the delta maths.** Once a game object has a body, move it with the body (velocity, acceleration), never by setting `x` and `y` - that teleports it past the collision checks. Chapter 8 said this; this chapter shows why the engine can do better
- **acceleration changes velocity; velocity changes position.** Students who confuse the two write `setAcceleration` for a walking player and wonder why it never stops
- **drag only works with zero acceleration** on that axis. It is the answer to half the "my ship never slows down" questions
- **damping inverts the drag number:** 0.9 is gentle, 0.1 is strong
- **Arcade bodies never rotate.** Rotation only turns the picture. Circles hide this; boxes do not. That, and stacking, is the case for Matter
- **the group trap:** add to the group first, then configure the body
- **angles start pointing right.** A picture drawn pointing up needs `rotation - Math.PI / 2`
- **Matter has its own units:** velocities per step, gravity 1, forces around 0.01

Live demo: the group trap

In `ch09_physics_playground`, `PlaygroundScene.addBall()`, move `this.things.add(ball);` from the top to the bottom of the method, build, and drop some balls. They fall straight down with no bounce and no sideways speed, and any that miss the crates and platforms drop straight through the floor (world bounds collision was reset too). There is no error. Ask students to find out why before showing slide 7. Put the line back and rebuild.

A second, shorter demo: in `Ship.thrust()`, delete `- Math.PI / 2`. The ship flies sideways.

Common problems and errors

What students see	Cause	Fix
A body added to a group does not move, has no bounce, or ignores world bounds; no error	the group reset it to its defaults when it was added	add it to the group first, then set
TS2551 [ERROR]: Property 'setAllowGravity' does not exist on type 'Body StaticBody'. Did you mean 'allowGravity'?	<code>this.body!.setAllowGravity(false)</code> - the body's type includes static bodies, which have no gravity	<code>(this.body as Phaser.Physics.Arcade.Body</code>
TS2551 [ERROR]: Property 'setAllowGravity' does not exist on type 'MovingPlatform'. Did you mean 'setGravity'?	calling a body-only method on the game object	call it on the body, as above
The ship never slows down	drag set, but the acceleration is never set back to 0	<code>setAcceleration(0)</code> when th
Damping stops everything instantly	<code>setDrag(0.05)</code> with damping on means "keep 5%"	use values near 1 (0.6 - 0.95) f
The ship flies sideways	the picture points up; angle 0 is right	subtract <code>Math.PI / 2</code> , or dra
The ship is faster diagonally	<code>setMaxVelocity</code> limits x and y separately	<code>body.setMaxSpeed(...)</code>
Objects vanish at the edge and pop back	<code>world.wrap</code> with no padding	pass a padding of about half th

What students see	Cause	Fix
Stacked Arcade crates jitter and sink	Arcade cannot really stack	expected - use Matter for stack
TS2345 [ERROR]: Argument of type 'Scene' is not assignable to parameter of type 'World'.	<code>super(scene, ...)</code> in a class extending <code>Phaser.Physics.Matter.Image</code>	Matter game objects take <code>scene</code>
<code>TypeError: Cannot read properties of undefined (reading 'add')</code> in <code>create</code> , in a Matter game	<code>this.physics.add...</code> when the config's default is "matter" - there is no Arcade plugin	use <code>this.matter.add...</code> (it c <code>this.physics</code> exists on every
Matter bodies fly off the screen	Arcade-sized numbers: <code>setVelocity(300, 0)</code>	Matter velocities are per step: 1
Debug drawing cannot be switched on later in Matter	bodies made while debug was off have no outline style	<code>debug: true</code> in the config, an the start of <code>create()</code> (as the j

Discussion questions

- Chapter 1's ball moved itself in `preUpdate` with `delta`. What would you have to add to it to get gravity? Bounce with a bounce factor? Drag? (A good way to show how much the engine does.)
- Why does drag only apply when there is no acceleration? What would a car game feel like if it applied all the time?
- Linear drag or damping - which suits a car? A spaceship? A puck on ice? A ball rolling on grass?
- In Arcade, friction belongs to the platform, not to the crate. Why might the designers have chosen that? What can Arcade friction not do?
- Why is the physics step fixed, when frames are not? What goes wrong in a game that moves by `delta` per frame when the computer is slow?
- When is Matter worth its cost? Name a game for each engine.

Extension ideas

- give the playground a wind: a key that sets `world.gravity.x` to a small value
- show the velocity of the ship as an arrow drawn with `Graphics` every frame
- use `this.physics.accelerateToObject` to make a homing missile that follows the ship
- set `fps: 30` in the playground config, turn gravity up to jupiter and watch collisions get rougher; then try `fps: 120` . Ask what it costs
- in Matter, build a chain or a pendulum with `this.matter.add.constraint` , or a Newton's cradle with `this.matter.add.newtonsCradle`

Assessment ideas

- Practical: "make a lunar lander: gravity 50, UP thrusts the way the lander points, land gently (`blocked.down` with a small `velocity.y`) or crash"
- Code reading: show `addBall()` with the group line moved to the end, and ask what the player will see and why
- Short answer: the difference between velocity and acceleration, and between linear drag and damping, with a sketch of speed against time for each
- Short answer: two reasons to choose Arcade over Matter, and one game where Matter is the right choice

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/` .

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment. All six were played with the test harness (key presses, and `eval` to read bodies' velocities, bounces, masses and angles) as well as type checked and linted.

1. Gravity switch

Project: `solutions/ch09_challenge_1_gravity_switch` (from `ch09_physics_playground`)

Goal: see that gravity is a vector - a strength *and* a direction - and that the world's gravity can be changed at any time.

`src/scenes/PlaygroundScene.ts`

```
interface Direction {
  name: string;
  x: number;
  y: number;
}

const DOWN: Direction = { name: "down", x: 0, y: 1 };
const UP: Direction = { name: "up", x: 0, y: -1 };
...
keyboard.on("keydown-LEFT", () => {
  this.setGravityDirection(LEFT);
});
...
private applyGravity(): void {
  const strength = GRAVITY_CHOICES[this.gravityIndex].value;
  this.physics.world.gravity.set(this.gravityDirection.x * strength,
  this.gravityDirection.y * strength);
}
```

The G handler now calls `applyGravity()` too, so a new strength keeps the chosen direction. Tested: LEFT gives gravity (-600, 0) and everything piles against the left wall; G

then gives (-1500, 0); UP gives (0, -1500), and things pile against the top edge and the underside of the ledge; R restarts with (0, 600) and "down".

Look for:

- **both** components set. The most common bug is `gravity.x = -600` for LEFT without zeroing `gravity.y`, so things fall diagonally
- the direction reset in `init()` - but note the world itself is rebuilt on restart, so the world gravity goes back to the config's value on its own; only the scene's field needs resetting
- the moving platform is unaffected (`allowGravity` is false) - worth asking students why

2. Bouncy and dead

Project: `solutions/ch09_challenge_2_bouncy_and_dead` (from `ch09_physics_playground`)

Goal: per-body settings, the group trap again, and a little inheritance.

A new class, `SpecialBall` extends `Ball`, remembers its own bounce and mass and applies them in `setUp()`; `Ball`'s constructor gains a default parameter for the picture.

`src/objects/SpecialBall.ts`

```
export class SpecialBall extends Ball {
  private readonly ownBounce: number;
  private readonly ownMass: number;

  constructor(scene: Phaser.Scene, x: number, y: number, key: string,
    bounce: number, mass: number) {
    super(scene, x, y, key);
    this.ownBounce = bounce;
    this.ownMass = mass;
  }

  public setUp(): void {
    this.setBounce(this.ownBounce);
    this.setMass(this.ownMass);
  }
}
```

src/scenes/PlaygroundScene.ts

```
// CHALLENGE 2: a SpecialBall is a Ball too, so ask about it FIRST - it
// keeps its own bounce
if (thing instanceof SpecialBall) {
    thing.setUp();
} else if (thing instanceof Ball) {
    thing.setBounce(BOUNCE_CHOICES[this.bounceIndex].value);
}
```

```
private addSpecialBall(x: number, y: number, key: string, bounce: number,
mass: number, tint: number): void {
    const ball = new SpecialBall(this, x, y, key, bounce, mass);
    ball.setTint(tint);
    this.things.add(ball);    // first - the group resets the body...
    this.applySettings(ball); // ...then drag, world bounds, and its own
    bounce and mass
}
```

Tested: after pressing 1, 2 and then B three times, the red balls have bounce 0 (dead), the blue ball still has bounce 1 and mass 0.5, and the grey ball bounce 0 and mass 5.

ball_blue.png is copied into public/assets/images/ and loaded in preload().

Look for:

- the order of the instanceof tests. With Ball first, the special balls match it and get B's bounce - a good discussion of "is-a" and the most specific type first
- setMass / setBounce called **after** this.things.add(...). Setting them in the constructor looks right and silently does nothing - the group wipes them
- simpler answers are fine: a keepsOwnBounce flag on Ball, or two plain images added to the group and set up in the scene. The key test is that B does not change them
- heavy on light: a grey ball dropped on a pile of blue ones scatters them; blue ones bounce off a grey one that hardly moves. Mass only matters between dynamic bodies

3. Brakes and reverse

Project: solutions/ch09_challenge_3_brakes_and_reverse (from ch09_space_ship)

Goal: understand that drag only applies with no acceleration, and that brakes are "just more damping".

src/objects/Ship.ts

```
// CHALLENGE 3: power 1 is full thrust forwards, -0.5 half thrust backwards,
// 0 none
public thrust(power: number): void {
  if (power !== 0) {
    ...
    this.scene.physics.velocityFromRotation(facing, THRUST * power,
    this.arcadeBody.acceleration);
  }
}
```

```
public brake(on: boolean): void {
  this.setDrag(on ? BRAKE_DAMPING : DAMPING);
}
```

with `BRAKE_DAMPING = 0.0001` : from 350 px/s, about 3.5 px/s is left after half a second ($350 \times 0.0001^{0.5}$).

src/scenes/SpaceScene.ts

```
const braking = this.brakeKey.isDown;
let power = 0;
if (this.cursors.up.isDown) {
  power = 1;
} else if (this.cursors.down.isDown) {
  power = -REVERSE_POWER;
}
this.ship.brake(braking);
this.ship.thrust(braking ? 0 : power);
```

Tested: full speed up (350), SHIFT held for 500 ms leaves 4 px/s; DOWN held for a second from rest reaches 147 px/s backwards (half thrust, less damping).

Look for:

- thrust switched **off** while braking. Students who only change the drag find that braking while holding UP does nothing - exactly the rule from the chapter. A great "why?" moment

- brake strength chosen by reasoning about damping (kept fraction per second), not by trial alone
 - `addKey(KeyCode.SHIFT)` or `this.cursors.shift` - both work
-

4. Rocks that split

Project: `solutions/ch09_challenge_4_rocks_that_split` (from `ch09_space_ship`)

Goal: a class with state (`size`), creating bodies inside a collision callback, directions from a velocity vector, and mass that finally matters.

`src/objects/Rock.ts`

```
export const BIG = 3;
export const MEDIUM = 2;
export const SMALL = 1;
const SCALES = [0, 0.7, 1.1, 1.5];           // indexed by size (index 0 is not
used)
const SPEED_UP = [0, 1.9, 1.4, 1];          // how much faster than a big rock
const POINTS = [0, 100, 50, 20];
...
public launch(angle: number = Phaser.Math.Between(0, 359)): void {
```

`launch` now takes a direction (a default parameter keeps the old call working), multiplies the speed by `SPEED_UP[this.size]`, and sets `setMass(this.size * this.size)`.

`src/scenes/SpaceScene.ts`

```

if (rock.size > SMALL) {
    const bulletBody = bullet.body as Phaser.Physics.Arcade.Body; // bullets
    have dynamic bodies
    const heading = Phaser.Math.RadToDeg(bulletBody.velocity.angle());
    for (const turn of [-SPLIT_ANGLE, SPLIT_ANGLE]) {
        // start each piece a little way along its own path, so the two do not
        overlap
        const nudge = this.physics.velocityFromAngle(heading + turn, SPLIT_GAP);
        const piece = new Rock(this, rock.x + nudge.x, rock.y + nudge.y,
rock.size - 1);
        this.rocks.add(piece);           // first: adding to a physics group
        resets the body...
        piece.launch(heading + turn);   // ...then set it moving
    }
}

```

The existing `countActive() === 0` check still starts the next wave, and now only fires when every piece is gone (the new pieces are added before the check). Tested: shooting every rock of wave 1 takes $4 + 8 + 16 = 28$ shots, scores $4 \times 20 + 8 \times 50 + 16 \times 100 = 2080$, and starts wave 2.

Look for:

- the pieces added to the group **before** `launch()`
- `velocity.angle()` (radians) converted for `velocityFromAngle` (degrees) - mixing the two units is the usual bug, and gives pieces flying off in odd directions
- pieces made at exactly the same point overlap and are pushed apart by the rocks' collider - it works, but looks odd; the nudge is a nicety
- the body follows the scale: `setScale` before `setCircle` in the constructor, radius in the picture's own pixels. (The body's size catches up with the scale by the next step)

Note for teachers - in the headless browser used to test these projects, rotated images sometimes draw with a wedge missing (a software-rendering quirk). In a normal browser the rocks draw whole; if students report "broken rocks", check they are not using a software renderer.

5. Demolition

Project: `solutions/ch09_challenge_5_demolition` (from `ch09_matter_stack`)

Goal: read the state of Matter bodies (angle, position) every frame to make a game rule.

`src/objects/Crate.ts`

```
// CHALLENGE 5: tipped over, or fallen? Matter bodies rotate, so angle (in
degrees, -180 to 180)
// tells us how far over it has gone
public get isDown(): boolean {
    return Math.abs(this.angle) > TIP_ANGLE || this.y > this.startY + DROP;
}
```

`src/scenes/StackScene.ts`

```
private fire(): void {
    // CHALLENGE 5: only SHOTS cannonballs; after the last, wait for things to
    settle, then finish
    if (this.shotsLeft === 0) {
        return;
    }
    this.shotsLeft = this.shotsLeft - 1;
    if (this.shotsLeft === 0) {
        this.time.delayedCall(SETTLE_TIME, () => {
            this.finished = true;
        });
    }
}
```

```
const down = this.targets.filter((crate) => crate.isDown).length;
```

`buildTower()` and `buildPyramid()` push each crate into `this.targets`, so crates dropped with 1 do not count; `init()` resets the array, the shots and `finished`, because R restarts the same scene object. Tested: five shots, then after four seconds "5 of 17 crates down"; R resets to "0 / 17, Shots left: 5".

Look for:

- `Math.abs(angle)` - a crate tipped the other way has a negative angle

- a crate lying on its side has angle 90 (or -90); one that has rolled right over may be back near 0, which is why the height test is there too
- counting every frame rather than once at the end (the score climbs as things fall - more fun)
- not counting before the stack has settled: Matter's crates wobble a little at the start

6. Pinball flipper

Project: `solutions/ch09_challenge_6_pinball_flipper` (from `ch09_matter_stack`)

Goal: a constraint, and driving a Matter body by its angular velocity.

`src/objects/Flipper.ts`

```
super(scene.matter.world, pinX + PIN_TO_CENTRE, pinY, FLIPPER_KEY,
undefined, {
  chamfer: { radius: THICKNESS / 2 }, // rounded ends, like the picture
  density: 0.05, // heavy: a ball landing on it
  // hardly moves it
  ignoreGravity: true, // it is held up by its pin and our
  // code, not gravity
  ignorePointer: true, // the mouse spring cannot drag it
  // off its pin
});
scene.add.existing(this);

// the pin: pointA is a place in the world, pointB a point on the body,
// measured from its
// centre (and turning with it). Length 0 and stiffness 1: the two are held
// exactly together
scene.matter.add.worldConstraint(this.body as MatterJS.BodyType, 0, 1, {
  pointA: { x: pinX, y: pinY },
  pointB: { x: -PIN_TO_CENTRE, y: 0 },
});
```

```

public swing(up: boolean): void {
    const target = up ? UP_ANGLE : DOWN_ANGLE;
    const speed = up ? FLIP_SPEED : RETURN_SPEED;
    const error = Phaser.Math.DegToRad(target - this.angle);
    this.setAngularVelocity(Phaser.Math.Clamp(error, -speed, speed));
}

```

The scene calls `this.flipper.swing(this.flipKey.isDown)` every frame. Because the angular velocity is never more than the angle still to go, the flipper stops exactly at its limits - the "check of its angle every frame" from the hint. A static peg just past the tip stops a ball rolling off the end. Tested: the flipper rests at 20 degrees and flips to -30 while Z is held; a ball dropped just above it rolls against the peg, and one flick sends it up and right into the tower, tipping four crates.

Look for:

- which way is "up". The tip is on the right, so lifting it is **anticlockwise** - a smaller angle. Students who pin the right end must reverse the signs; the ball then flies up and *left*
- Matter's `setAngularVelocity` is radians per step: 0.1 is brisk; 1 flings the ball off the top of the world (we tried 0.35 - the ball left the screen)
- `ignoreGravity` on the flipper, or it sags and fights the code; `ignorePointer`, or the mouse spring pulls it off its pin
- static stoppers instead of the angle check are a fine answer too - the flipper bounces against them a little, which looks quite like a real flipper
- the red ball is bouncy (restitution 0.7): dropped from high up, it bounces off the flipper and over the peg. Dropping it just above the flipper, or lowering its restitution, fixes that

Slides

Chapter 9

2D physics

Velocity, gravity, drag, bounce - and a spaceship



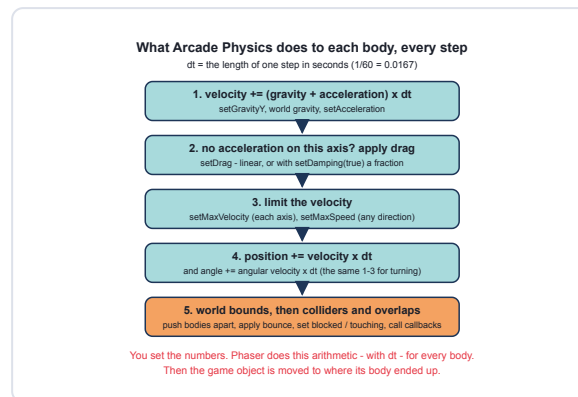
1 / 20

Today

- what the physics engine does to every body, every step
- gravity, bounce, drag and damping, mass, friction
- kinds of body: dynamic, immovable, static
- a trap: groups reset their bodies
- turning, thrust, top speed, wrapping round the world
- Matter.js: shapes that rotate and stack

2 / 20

Every body, every step



You set the numbers. Phaser does the `delta` maths.

3 / 20

Velocity and acceleration

- **velocity** - pixels per second. Changes the **position**
- **acceleration** - pixels per second, per second. Changes the **velocity**
- **gravity** - an acceleration every body shares

`setVelocityX(200)` - right at 200 px/s, from now on

`setAccelerationX(200)` - 200 px/s faster every second: 600 px/s after three seconds

4 / 20

Gravity and bounce

```
// G: the world's gravity - shared by every body (a body can add its own with setGravityY)
keyboard.on("keydown-G", () => {
  this.gravityIndex = (this.gravityIndex + 1) % GRAVITY_CHOICES.length;
  this.physics.world.gravity.y = GRAVITY_CHOICES[this.gravityIndex].value;
});
```

```
// the physics world starts as big as the game; stop it at the top of the ground instead
this.physics.world.setBounds(0, 0, 800, FLOOR_Y);
```

- `setBounce(b)` : 0 dead, 1 perfect, more than 1 gains speed
- `setCollideWorldBounds(true)` : the world's edges are walls

5 / 20

Drag, two ways



- `setDrag(150)` - lose 150 px/s every second
- `setDamping(true)` + `setDrag(0.3)` - keep 30% every second
- **drag only works when acceleration is 0**

6 / 20

The trap: groups reset their bodies

```
private addBall(x: number, y: number): void {  
  const ball = new Ball(this, x, y);  
  // add to the group FIRST: a physics group resets the bodies it is given to its own defaults  
  // (no bounce, no velocity...), so anything set before this line would be lost  
  this.things.add(ball);  
  bodyOf(ball).onWorldBounds = true;  
  ball.setVelocityX(Phaser.Math.Between(-LAUNCH_SPEED, LAUNCH_SPEED));  
  this.applySettings(ball);  
}
```

No error. No warning. The ball just stops.

7 / 20

Kinds of body

Body	Moves itself?	Pushed?
dynamic	yes	yes, shared by mass
<code>setPushable(false)</code>	yes	no - pushes back
<code>setImmovable(true)</code>	if given a velocity	never
static	never	never (cheapest)

Arcade **friction** belongs to a moving immovable body: how much of its movement it passes to a rider.

`setFriction(1, 0)` carries crates; `0` slides out from under them.

8 / 20

What is it touching?

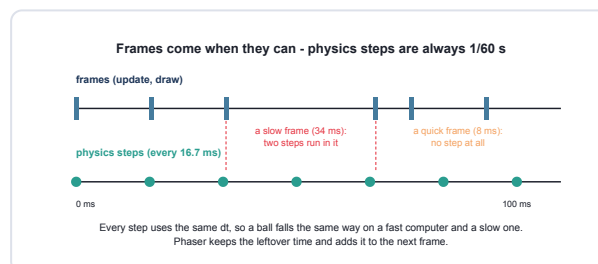
- `body.blocked.down` - resting on something that cannot give way
- `body.touching.down` - touching any body below
- worked out afresh every step: check in `update()`

```
this.physics.world.on(
  Phaser.Physics.Arcade.Events.WORLD_BOUNDS,
  (body: Phaser.Physics.Arcade.Body, _up: boolean, down: boolean) => {
    // ...
  },
);
```

Only for bodies with `onWorldBounds = true`

9 / 20

The physics clock

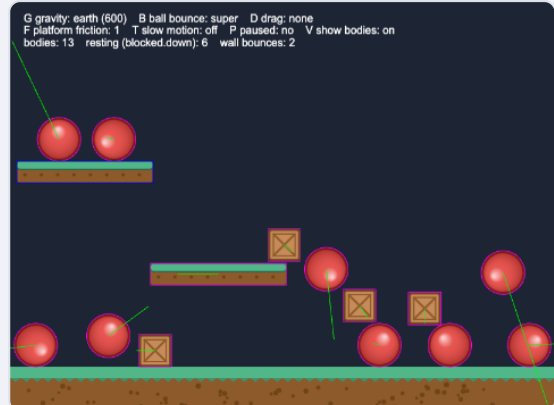


- `this.physics.pause()` / `resume()` - the scene keeps running
- `world.timeScale = 3` - slow motion (**bigger is slower**)

10 / 20

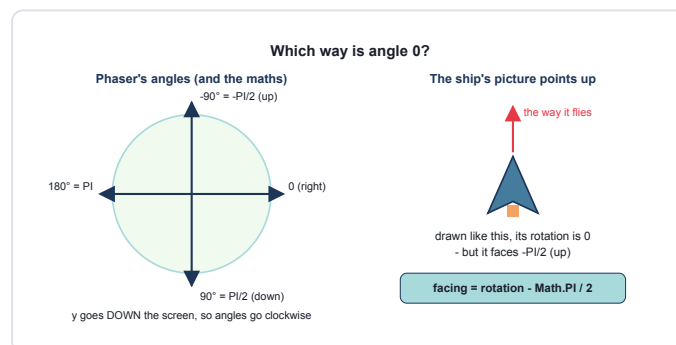
Try it: the playground

- G B D F T P - change the laws of physics
- V - see the real bodies
- watch a ball roll: the picture turns, the **body never does**
- drop a crate on the platform; press F



11 / 20

Which way is angle 0?



12 / 20

Turning and thrust

```
public turn(direction: number): void {
    this.setAngularAcceleration(direction * TURN_ACCELERATION);
}

public thrust(on: boolean): void {
    if (on) {
        ...
        const facing = this.rotation - Math.PI / 2;
        ...
        this.scene.physics.velocityFromRotation(facing, THRUST, this.arcadeBody.acceleration);
    } else {
        ...
        this.setAcceleration(0);
    }
}
```

13 / 20

Drift, top speed, wrap

```
this.setDamping(true);
this.setDrag(DAMPING);
...
this.arcadeBody.setMaxSpeed(MAX_SPEED);
```

DAMPING = 0.6 - keep 60% a second. setMaxSpeed - in any direction

- setMaxVelocity(x, y) limits each axis: diagonals go 1.4 times faster

```
this.physics.world.wrap(this.ship, SHIP_WRAP);
this.physics.world.wrap(this.rock, ROCK_WRAP);
```

14 / 20

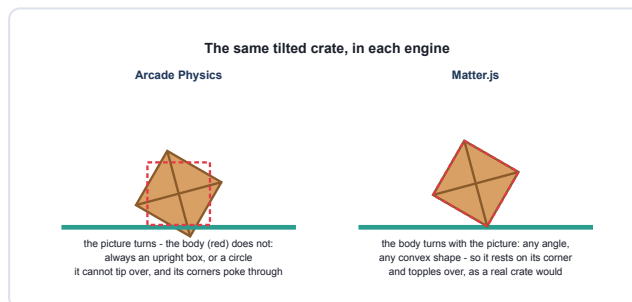
Try it: the space ship

- LEFT / RIGHT turn, UP thrusts, SPACE fires
- turn round and thrust to stop
- bullets get the ship's velocity added
- `this.physics.pause()` at game over



15 / 20

Matter.js: why a second engine?



```
physics: {  
  default: "matter",  
  matter: {  
    ...  
    gravity: { x: 0, y: 1 },  
  },  
}
```

16 / 20

Matter game objects

```
export class Crate extends Phaser.Physics.Matter.Image {  
  constructor(scene: Phaser.Scene, x: number, y: number) {  
    super(scene.matter.world, x, y, CRATE_KEY, undefined, {  
      friction: 0.6,          // wood on wood: grippy, so a stack stays up (0 = ice, 1 = rubber)  
      restitution: 0.05,      // bounce: crates hardly bounce at all  
    });  
    scene.add.existing(this);  
  }  
}
```

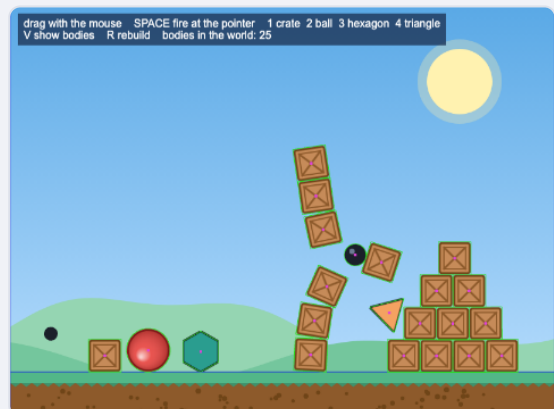
- `shape: { type: "circle", radius }, { type: "polygon", sides, radius }`
- `density` gives mass; velocities are **pixels per step**

17 / 20

Try it: knock it down

```
this.matter.add.mouseSpring({ stiffness: 0.2 });
```

- drag anything; SPACE fires at the pointer
- 1 - 4 drop shapes; V shows them
- Arcade for most games; Matter when the game is the physics



18 / 20

Summary

- set velocity and acceleration - the engine integrates, in fixed steps
- gravity, bounce, drag (linear or damping), max speed, mass, friction
- dynamic, immovable, static; `blocked` , `touching` , `"worldbounds"`
- groups reset bodies: **add first, then set up**
- angular velocity and acceleration; `velocityFromRotation` ; `world.wrap`
- Matter: real shapes, rotation, stacking - in its own units

19 / 20

Challenges

1. **Gravity switch** - arrow keys choose which way gravity pulls
2. **Bouncy and dead** - a light super-ball and a heavy dead one
3. **Brakes and reverse** - DOWN for retro-rockets, SHIFT to brake
4. **Rocks that split** - big to medium to small
5. **Demolition** - five cannonballs, count the crates down
6. **Pinball flipper** - a pinned Matter body, flicked with Z

Next: **Chapter 10 - Tilemaps**

20 / 20